

B4B33PSY: Seminar 2

Bc. Štěpán Pressl

Podmínky v Bashi

- Syntax podmínky je následující:

```
if <návratová-hodnota-programu>; then
    # vykoná se pokud program skončil 0
else
    # pokud jinak
fi
```

- Oneliner: `if <podm>; then <cmd1>; else <cmd2>; fi;`
- Existuje `true` a `false` - nejedná se o speciální výrazy, ale o programy které vracejí 0/1.

Příklady

Příklad 1

- Vyzkoušíme si příkaz `test`, který umožňuje zapsat podmínky v Bashi. Nejdřív si vytvoříte složku `~/psy/cv2` a v ní soubor `skript1.sh`. Vyzkoušíme si několik příkazů a zkusíme si napsat první skript

Příklad 2 (relativní cesta)

- Otestuje, že soubor `skript.sh` existuje relativně ve stejném adresáři, kde se nachází.
 - Náповěda: skript se dá pustit odkudkoliv, pokud znáte jeho cestu. Je potřeba ale vázat cestu relativně k umístění skriptu. Na zjištění na to existuje následující příkaz:

```
absdir=$(cd $(dirname "$0"); pwd)
```

Magický příkaz

`set -euo pipefail`

- -e: ukončení skriptu při jakékoli chybě příkazu
- -u: ukočení skriptu při nedefinované proměnné
- -o: chyba uvnitř pipe (vysvětlíme příště)

Životnost proměnných a jak se spouští skripty

- Skripty můžete pouštět pomocí jejich cesty, je nutno změnit práva. Spustí se v subshellu.
- Skripty můžete pouštět přes `bash skript.sh`. Není potřeba měnit práva. Spustí se v subshellu.
- Lze dát více příkazů jednoho vykonávání
 - `(cd adresar && mkdir podadresar)` - vytvoří se subshell a tam se tomto spustí
 - `{cd adresar && mkdir podadresar}` - spustí se v momentálním shellu
- `source skript.sh` - všechny řádky skriptu se vykonají v momentálním shellu
- `export PROM` exportujte proměnné do subshellů, je nutno použít pokud voláte (...) či spouštíte skripty, které vyžadují nějaké proměnné od rodičovského shellu
- platí pravidlo, že proměnné v subshellu se nepředávají do rodičovského

Proč to říkám

- Je několik příkazů, na které budete často narážet
 1. `source .venv/bin/activate`
 - Aktivace takzvaného virtuálního environmentu v Pythonu.
 2. `source ~/.bashrc`
 - `~/.bashrc` je konfigurace BASH shellu pro vašeho usera. Aplikace změn vyžaduje zavolat `source`
 3. Systémové proměnné (např. `$PATH`)
 - `$PATH` jsou cesty k programům, které tento shell zná, oddělené dvojtečkou
 - předává se vždy dál, při modifikaci (typicky v `~/.bashrc`) je potřeba nastavit přes `export`!

Více k proměnným

- Speciální proměnné do skriptu:
 - \$1, \$2, ..., \$9, \${10} - poziční argumenty
 - \$# počet předaných argumentů
 - \$@ - pole předaných argumentů
- Zachycení výstupu skriptu
 - \$(prikaz), třeba \$(pwd).

Cykly v Bashi

- V Bashi existují dva základní cykly: `for` a `while`.
- `for` iteruje přes **seznam slov**, `while` opakuje tělo, dokud podmínka (tj. návratová hodnota programu) vrací 0. Dají se použít `break` a `continue`.

```
for promenna in seznam slov; do
    echo "$promenna"
done
```

```
while <návratová-hodnota-programu>; do
    # opakuje se, dokud program vrací 0
done
```

Cyklus for

- Seznam se dá zapsat výčtem, rozsahem, nebo expanzí:

```
for i in a b c; do echo "$i"; done
```

```
for i in {1..10}; do echo "$i"; done           # rozsah
```

```
for f in *.txt; do echo "$f"; done            # globbing
```

```
for u in $(cat users.txt); do echo "$u"; done
```

Cyklus while

- Typické použití - čtení souboru po řádcích:

```
while read -r radek; do
    echo "Řádek: $radek"
done < soubor.txt
```

- Přepínač -r zakáže interpretaci zpětných lomítek, tj. \t zůstane \t a nestane se z něj tabulátor.
- Nekonečný cyklus s podmínkou uvnitř:

```
while true; do
    read -r odpoved
    if [ "$odpoved" = "konec" ]; then break; fi
done
```

Proměnná IFS

- IFS (Internal Field Separator) říká Bashi, podle jakých znaků má dělit text na jednotlivá slova (pole).

```
for f in $(echo "muj soubor.txt jiny.txt"); do echo  
"$f"; done
```

- Hodí se v případě, kdy máte v proměnné nějaký separátor. Vždy si uložte \$IFS do \$OLDIFS a poté udělejte restore

Příklady

Příklad 3

- Napište skript, který projde všechny soubory *.sh v adresáři skriptu a u každého vypíše, zda je spustitelný (test -x).

Příklad 4

- Napište skript, který po řádcích přečte soubor a očísluje jeho řádky (bez použití cat -n). Pro inkrementaci proměnné použijte

```
line=$(( line+1 ))
```